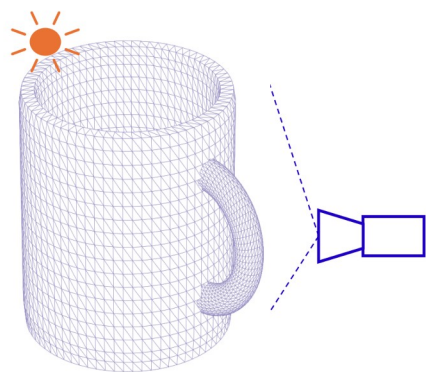




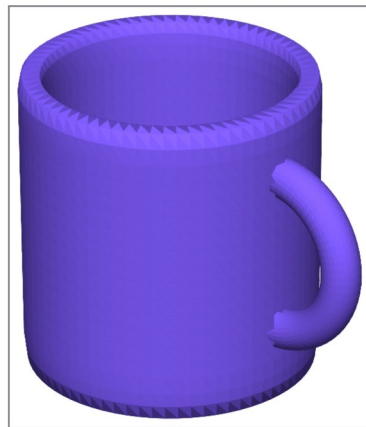
3D компьютерное зрение и графика

Лекция 2. Геометрия: от линейной алгебры к $SO(3)$

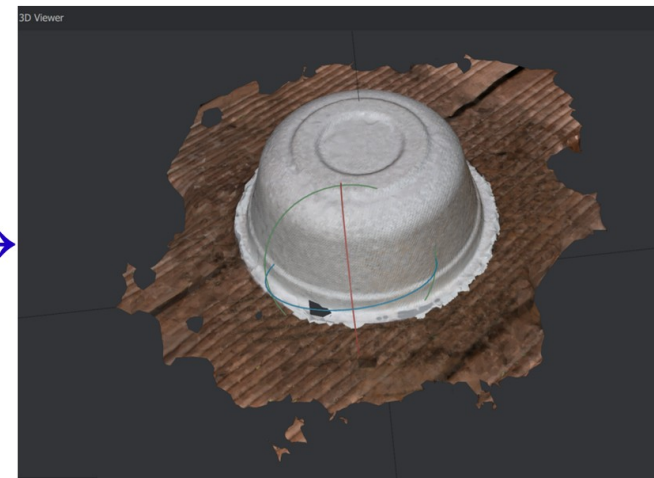
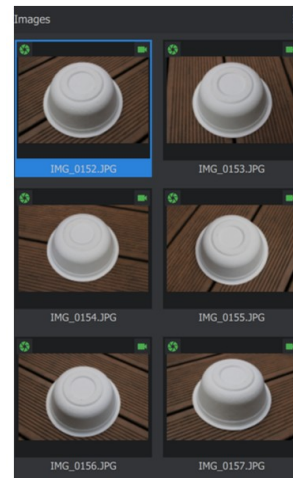
Жемчужников Дмитрий Сергеевич



геометрия, материалы, свет, камера



изображение



Прямая: сцена → изображение

рендеринг; определена однозначно

лекции 6–8

Обратная: изображение → сцена

реконструкция; ответ не единственный

лекции 3–5 и вторая половина курса

слева — собственная отрисовка; справа — скриншот Meshroom, Colin Kranz, CC BY-SA 4.0, Wikimedia Commons

Это не две дисциплины рядом, а одна задача с двух сторон

во второй половине курса они сомкнутся: прямую задачу сделают дифференцируемой и через неё будут решать обратную





настоящая трещина в леднике



краска на ровном асфальте

Одному изображению отвечает бесконечно много сцен

- это свойство задачи, а не нехватка вычислений
- выбрать один ответ можно только по знанию, которого в изображении нет



Три неоднозначности: чем именно недоопределена задача



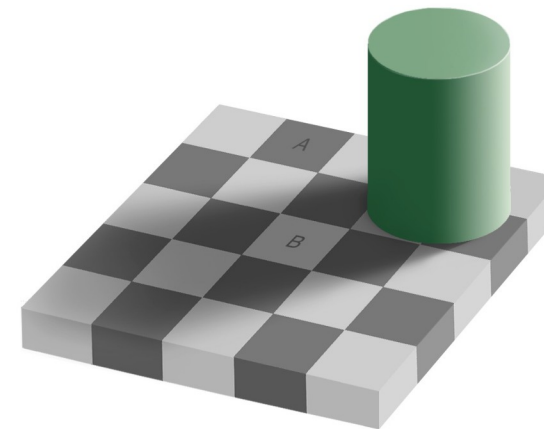
глубина вдоль луча

хранит направление, а не дальность;
одна женщина — в обоих углах



масштаб

увеличим сцену и базу в s раз —
снимки получатся те же



материал против освещения

яркость = альбеда × освещённость;
клетки A и B одного оттенка

комната Эймса — Sophie Wiesinger, CC BY-SA 4.0 · Тадж-Махал — Buiobuione, CC BY-SA 4.0 · шахматная иллюзия — Edward H. Adelson, файл Gustavb

Ни одна из трёх не снимается лучшей камерой: это свойство самой задачи

- инструментальные искажения — отдельная история: объектив гнёт прямые, но дисторсию можно измерить и убрать (лекция 3)
- поэтому изображение — не измерение сцены, а её проекция с потерями



Чем доопределяют: три источника



больше наблюдений

второй кадр даёт второй луч

лекции 4, 5



физические приоры

гладкость, ламбертовость,
манхэттенский мир

лекция 3



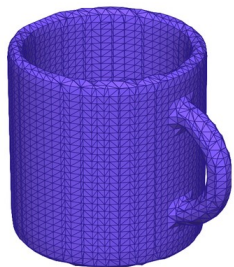
выученные приоры

статистика, извлечённая
из данных
лекции 9–11

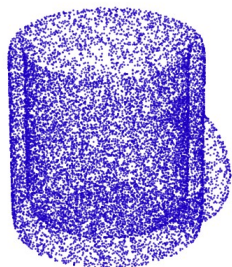
Ошибки у трёх источников разные — и это подсказывает, кто подвёл

сегодня занимаемся первым: чтобы соединить два наблюдения, надо уметь записать, где они сделаны друг относительно друга

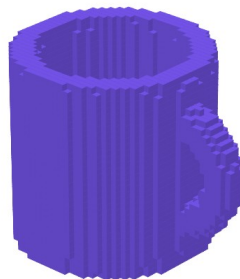




полигональная сетка
вершины и грани



облако точек
множество без связей



воксели
регулярная 3D-сетка



неявная функция
поверхность = $\{ f = 0 \}$



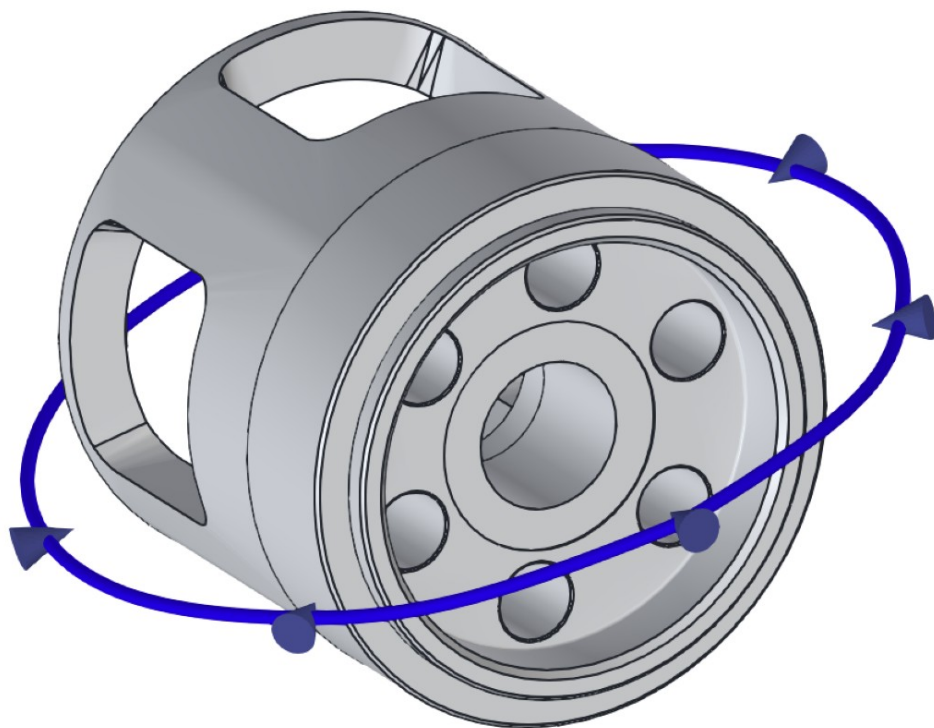
гауссовы сплаты
примитивы с объёмом

Представление — это структура данных плюс операции, которые она делает дешёвыми

- универсального представления нет: выбор — это выбор того, что будет дешевле
- сегодня нам нужно одно представление — облако точек



Как получают трёхмерную модель детали



траектория сенсора вокруг детали



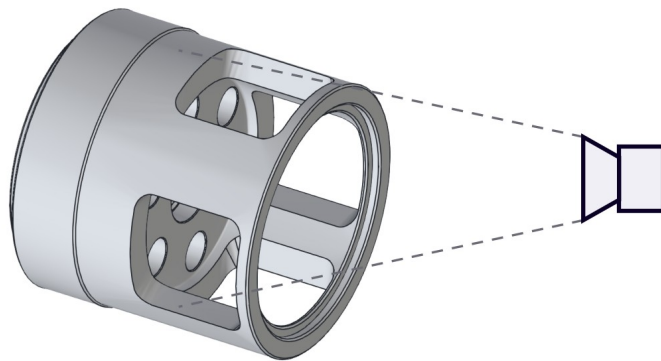
облако точек всей детали

Сенсор меряет только то, что видит, поэтому его водят вокруг детали

все проходы вместе закрывают поверхность целиком — это и есть исходные данные для сетки и CAD-модели



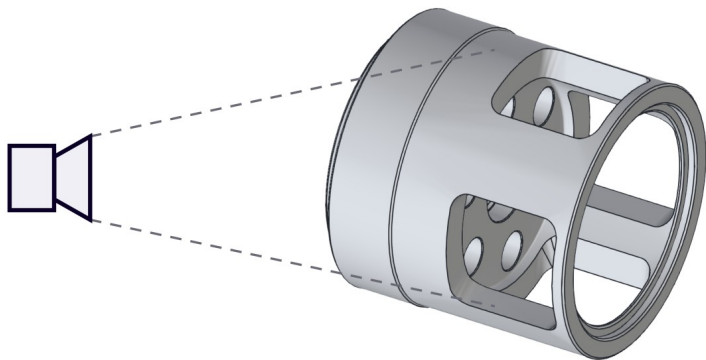
Один проход видит только одну сторону



модель и сенсор: первый проход



что измерил этот проход



модель и сенсор: второй проход



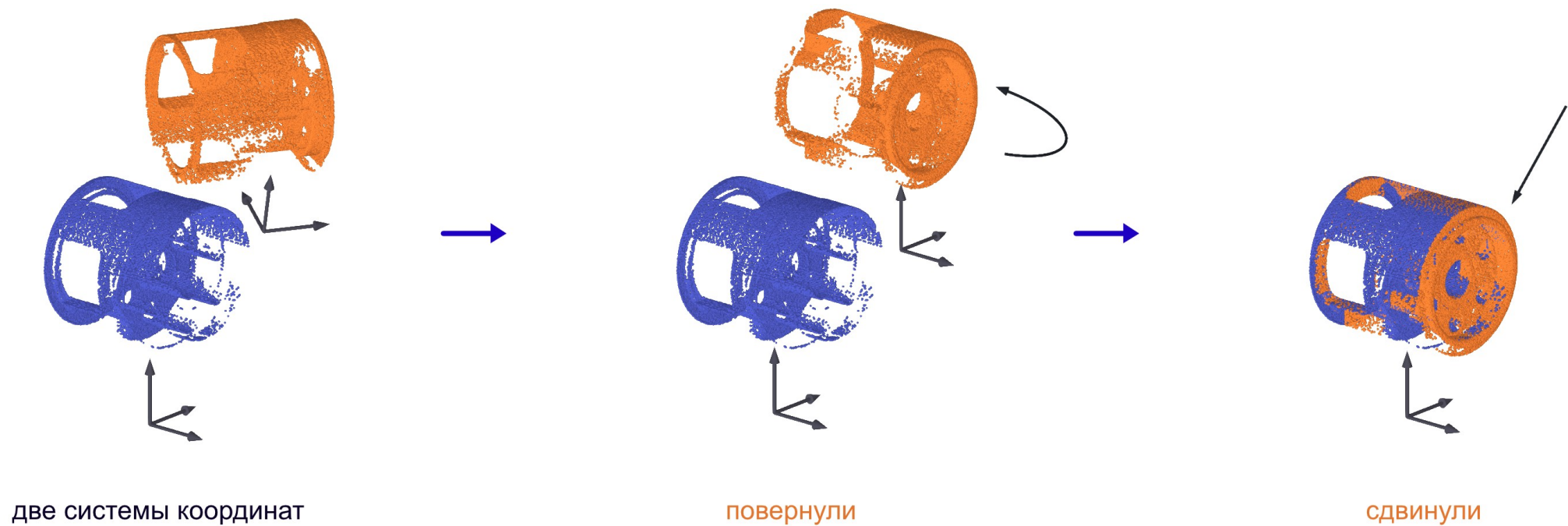
что измерил этот проход

За один проход измеряется около половины поверхности: здесь 44 % и 48 %

проходы перекрываются — без общего куска их нечем будет связать



Нам нужно совместить два скана

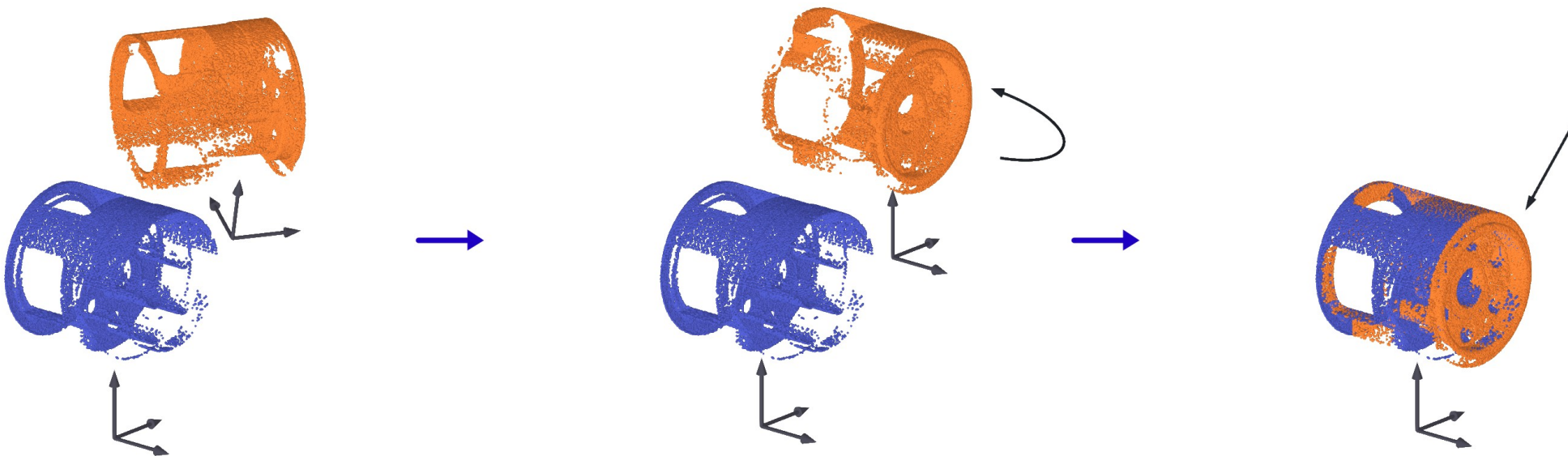


Совместить — значит повернуть второй скан и сдвинуть его на первый

поворот и сдвиг вместе и есть то преобразование, которое нам нужно найти



Что значит повернуть и сдвинуть



две системы координат

повернули

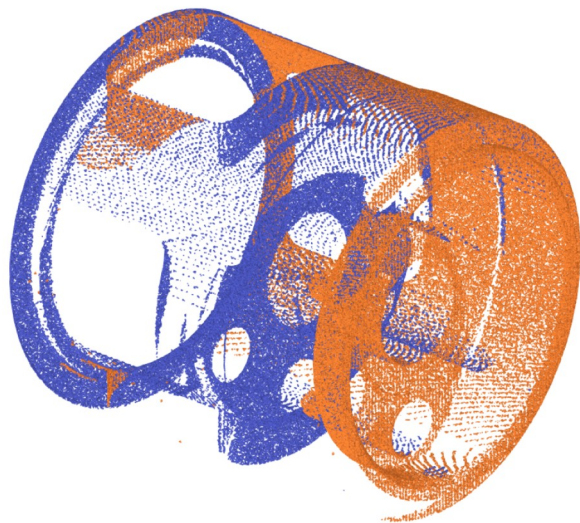
сдвинули

$$\mathbf{x} \mapsto R\mathbf{x} \mapsto R\mathbf{x} + \mathbf{t}$$

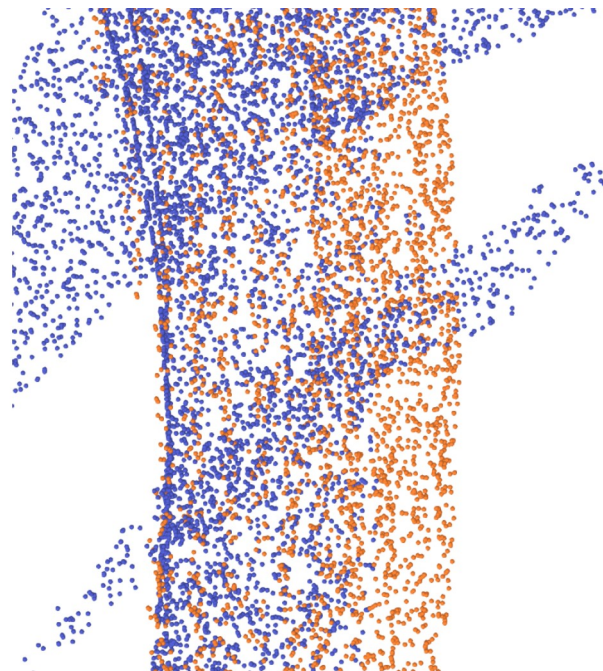
Совместить два скана — значит найти R и $\mathbf{t}$



Почему это считают, а не подгоняют мышкой



ошибка в 1° : на экране её не видно



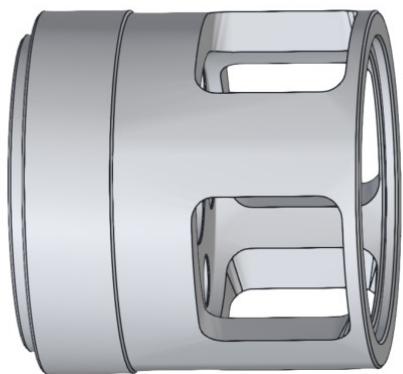
та же кромка вблизи — 1.3 мм

и проходов не два, а полтора десятка — ошибка каждой стыковки копится

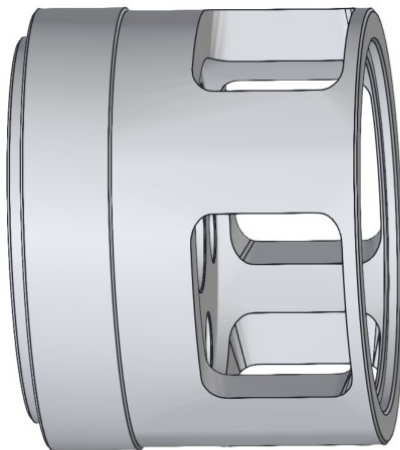
Глаз не различает того, что различает измерение

поэтому нужен критерий и алгоритм, а не мышка

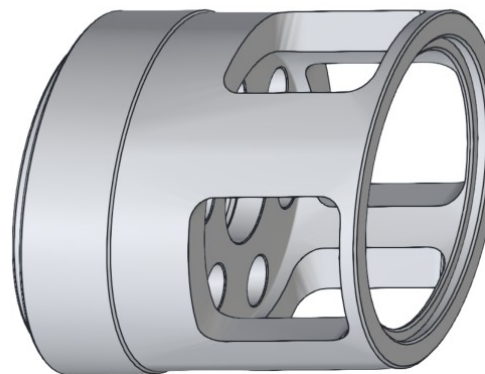




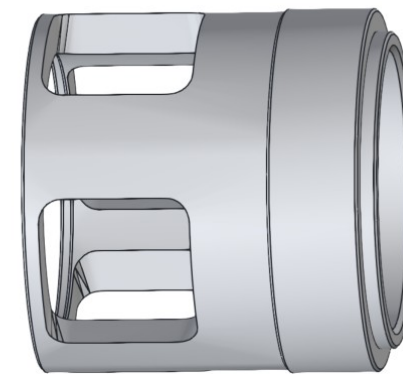
жёсткое движение



растяжение



скос



зеркало

- деталь между проходами не изменилась: длины и углы сохранились
- растяжение, скос и зеркало означали бы, что изменилась сама деталь

Значит, ищем движение твёрдого тела: поворот и сдвиг, и ничего больше

зеркало выглядит безобиднее прочих, а поймать его труднее всего — вернёмся к нему дважды



Чем записать

однородные координаты
и группа $SE(3)$

Как найти

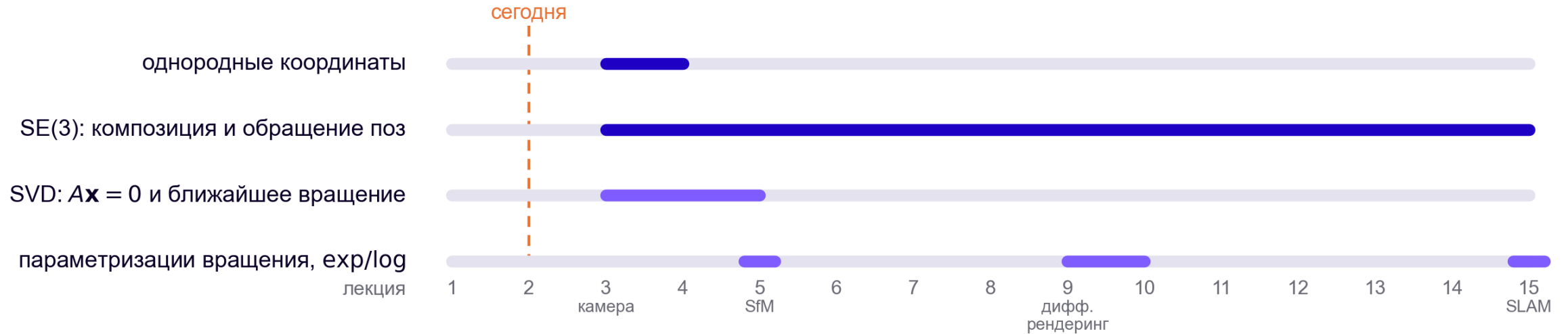
SVD и алгоритм Кабша

Как хранить

$SO(3)$, ось-угол,
кватернионы

Как уточнить

приращение
в касательном
пространстве

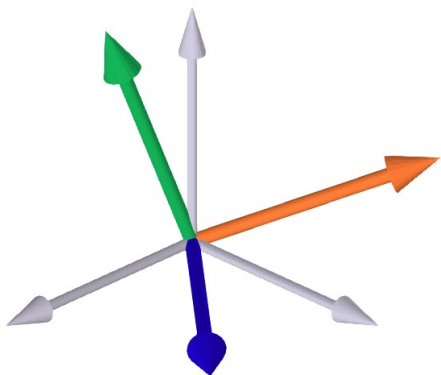


Единственная лекция курса, где ничего не реконструируется

зато всё, что здесь появится, работает дальше каждую неделю



Три объекта: точка, поворот, сдвиг



$$R = \begin{bmatrix} +0.66 & -0.70 & +0.29 \\ +0.75 & +0.64 & -0.17 \\ -0.07 & +0.33 & +0.94 \end{bmatrix}$$

столбцы — образы базисных векторов

x

координаты точки: три числа

R

матрица поворота 3×3

t

вектор сдвига: три числа

$$\mathbf{x} \mapsto R\mathbf{x} + \mathbf{t}$$

Столбцы единичны и попарно ортогональны

одним равенством: $R^T R = I$

это и значит «сохраняет длины и углы»

Определитель равен +1

из ортогональности следует только $\det R = \pm 1$

минус единица — зеркало со слайда 12

Матрица поворота — не любые девять чисел: на них два условия

первое условие ловит растяжение и скос, второе — зеркало



$$R_2(R_1\mathbf{X} + \mathbf{t}_1) + \mathbf{t}_2 = R_2R_1\mathbf{X} + R_2\mathbf{t}_1 + \mathbf{t}_2$$

свернуть два движения в одно: вращения перемножились, а переносы — нет

Сканов не два, а десяток

на слайде про обход их было
четырнадцать
движения придётся сцеплять в цепочку
правило сцепления несимметрично и
держится в голове

Движение надо обращать

«скан 2 в систему скана 1» и наоборот
обращение — ещё одна формула, и в
ней не просто минус t

У пары (R, t) нет своего типа

умножение для неё не определено
нужен свой класс со своим сцеплением

Хочется, чтобы сцепление было умножением, а обращение — обратной матрицей

мешает одно: матрицы 3×3 для сдвига не существует — линейное отображение обязано переводить ноль в ноль



Оба действия — в одной матрице

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$

$$T = \begin{bmatrix} R & \mathbf{t} \\ 0^\top & 1 \end{bmatrix} \quad \mathbf{p}' = R\mathbf{p} + \mathbf{t}$$

на плоскости: приписали единицу — и сдвиг стал умножением

в пространстве: R и $\mathbf{t}$ в одной матрице 4×4

В чём приём

к координатам точки приписываем единицу

матрица становится на строку и столбец больше

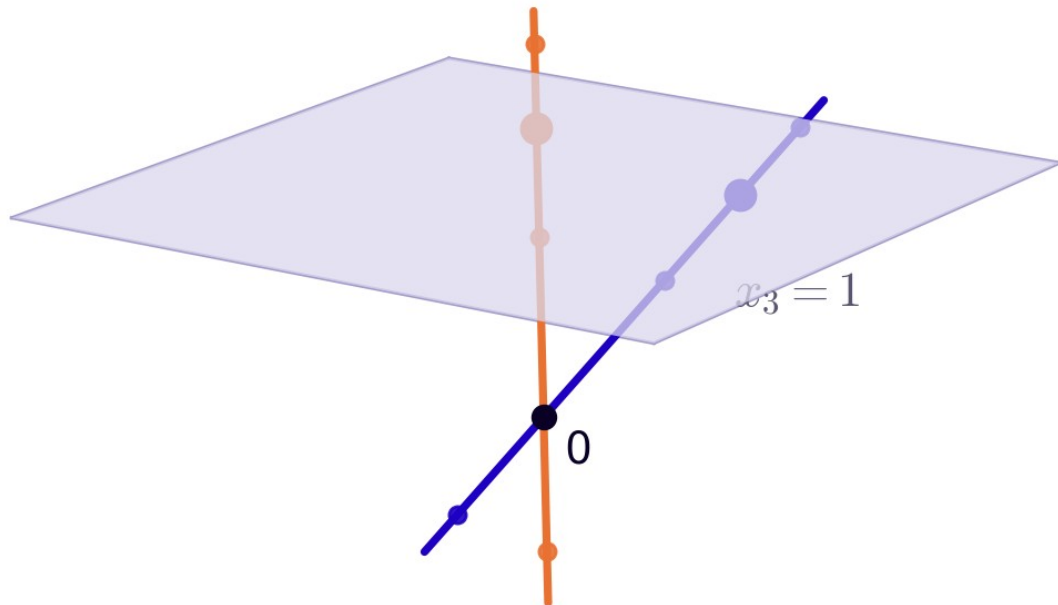
последняя строка $(0, \dots, 0, 1)$ следит, чтобы единица осталась единицей

Оба действия уместились в одно матричное умножение

обратно в трёхмерии — делением на последнюю координату; что означает приписанная единица и почему делить можно — на следующем слайде



Что означает приписанная единица: однородные координаты



$$\tilde{\mathbf{x}} \sim \lambda \tilde{\mathbf{x}}, \quad \lambda \neq 0$$

Обратно в трёхмерие

$$(x_1, x_2, x_3, x_4) \mapsto \left(\frac{x_1}{x_4}, \frac{x_2}{x_4}, \frac{x_3}{x_4} \right), \quad x_4 \neq 0$$

Все ненулевые векторы одной прямой через ноль — это одна точка



жёсткое движение

$$\begin{bmatrix} & R & \mathbf{t} & \\ & & & \\ & & & \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{1} \end{bmatrix} \cdot \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \Rightarrow w' = 1$$

делить не на что

камера

$$\begin{bmatrix} & f & c & \\ & & & \\ & & & \\ \mathbf{0} & \mathbf{0} & \mathbf{1} & \mathbf{0} \end{bmatrix} \cdot \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \Rightarrow w' = Z$$

делим на глубину:
перспектива

Камера отличается от жёсткого движения одной строкой матрицы



$$T_2 T_1 = \begin{bmatrix} R_2 R_1 & R_2 \mathbf{t}_1 + \mathbf{t}_2 \\ 0^\top & 1 \end{bmatrix}$$

композиция — обычное произведение;
читается справа налево: сначала T_1

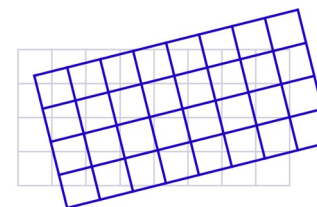
$$T^{-1} = \begin{bmatrix} R^\top & -R^\top \mathbf{t} \\ 0^\top & 1 \end{bmatrix}$$

обратный перенос $-R^\top \mathbf{t}$, а не $-\mathbf{t}$:
вектор сначала поворачивают

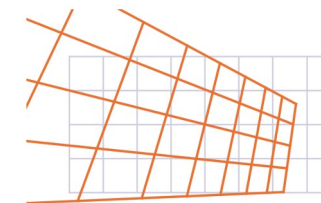
Ошибка, которая не падает: $T_{\text{inv}} = T.T$

столбец $\mathbf{t}$ уезжает в последнюю строку

жёсткое движение стало проективным, а исключения не было



верно: жёсткое движение



а так делает $T^\top$

Композиция стала умножением, обращение — двумя транспонированиями



SO(3) — special orthogonal

orthogonal — $R^T R = I$: длины и углы сохранены

special — $\det R = +1$: отражения исключены

SE(3) — special Euclidean

Euclidean — поворот и сдвиг евклидова пространства

матрицы 4×4 из $R \in \text{SO}(3)$ и произвольного t

12 значащих чисел

девять в R и три в t

— **6** уравнений

из $R^T R = I$: три на длины и три на ортогональность

= **6** степеней свободы

три вращательных и три поступательных

Преобразование между сканами — элемент SE(3): шесть чисел в матрице 4×4



$$T_{13} = T_{12} T_{23}, \quad T_{21} = T_{12}^{-1}$$

читаем справа налево: правый индекс — откуда, левый — куда; внутренние индексы сокращаются

Поза: из сенсора в мир

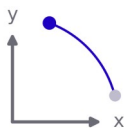
последний столбец — положение сенсора в мире
столбцы R — оси сенсора в мировых координатах

Матрица вида: из мира в сенсор

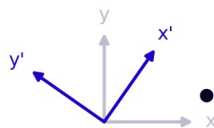
её перенос центром НЕ является
центр равен $-R^T t$ — то же обращение, что на слайде 19

Держите направление в имени переменной: `T_scan1_scan2`, а не `pose`

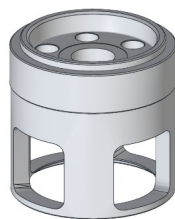




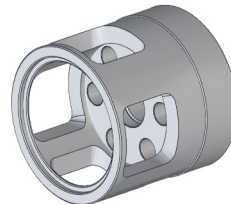
активно: повернулась точка



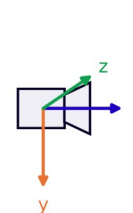
пассивно: повернулись оси



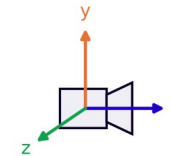
сначала поворот вокруг z



сначала поворот вокруг x



OpenCV: y вниз, z вперёд



OpenGL: y вверх, z назад

Активное или пассивное

точка повернулась — или мы сменили базис

записи отличаются транспонированием

Порядок и раскладка

у нас столбцы и действие слева: $T_2 T_1$ — сначала T_1

glTF хранит по столбцам, чтение построчно транспонирует

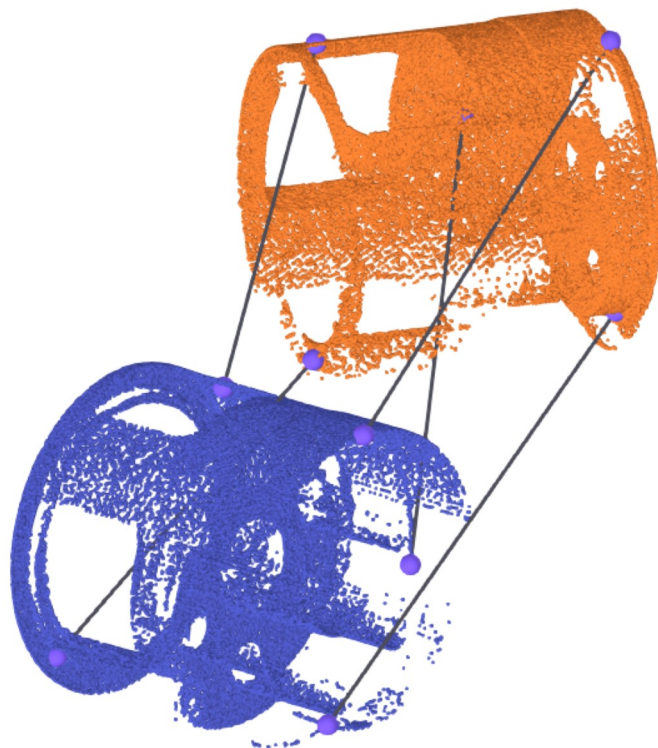
Куда смотрят оси

переход — $\text{diag}(1, -1, -1)$, и это законный поворот

флип одной оси — уже отражение

Пропала или дырявая — нечётное число флипов; перевёрнутая, но целая — чётное





5 пар: одно и то же место детали в двух системах координат

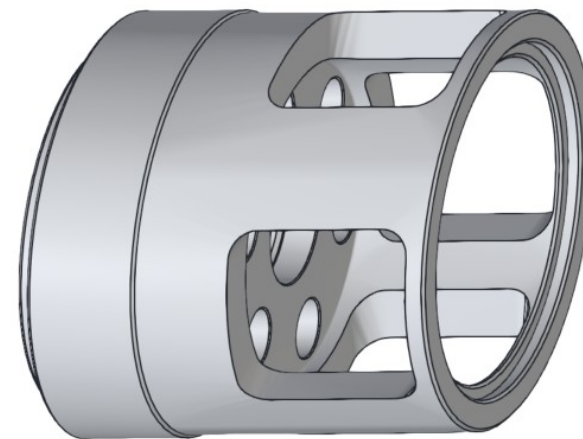
- в каждой паре — одна и та же физическая точка: её координаты в системе первого скана и в системе второго
- длина отрезка — то, насколько мы пока промахиваемся
- трёх пар достаточно, чтобы задача имела единственное решение
- найти надо R и t , при которых суммарный промах наименьший

Задача из геометрической стала задачей оценивания

минимум по t взять несложно, а R обязана быть поворотом: девять чисел, связанных шестью уравнениями



- девять чисел, квадратичный функционал, решение в замкнутой форме
- невязка честно меньше, чем у любого поворота
- но $R^T R \neq I$: деталь ещё и растянута и скошена



СКОС

Ограничение $R \in SO(3)$ надо учитывать в самой постановке



$$E(R, \mathbf{t}) = \sum_i \|R\mathbf{p}_i + \mathbf{t} - \mathbf{q}_i\|^2 \rightarrow \min, R \in SO(3)$$

$$\frac{\partial E}{\partial \mathbf{t}} = 2 \sum_i (R\mathbf{p}_i + \mathbf{t} - \mathbf{q}_i) = 0$$

$$\mathbf{t} = \bar{\mathbf{q}} - R\bar{\mathbf{p}}$$

при любом R оптимальный сдвиг совмещает центры масс

$$E(R) = \sum_i \|R\tilde{\mathbf{p}}_i - \tilde{\mathbf{q}}_i\|^2, \quad \tilde{\mathbf{p}}_i = \mathbf{p}_i - \bar{\mathbf{p}}, \quad \tilde{\mathbf{q}}_i = \mathbf{q}_i - \bar{\mathbf{q}}$$

Подставили $\mathbf{t}$ обратно — осталась одна неизвестная, поворот



$$E(R) = \sum_i \|\tilde{\mathbf{p}}_i\|^2 + \sum_i \|\tilde{\mathbf{q}}_i\|^2 - 2 \sum_i \tilde{\mathbf{q}}_i^\top R \tilde{\mathbf{p}}_i$$

серые слагаемые от R не зависят: поворот не меняет длин

$$\sum_i \tilde{\mathbf{q}}_i^\top R \tilde{\mathbf{p}}_i = \text{tr}(R^\top H) \longrightarrow \max$$

число равно следу матрицы 1×1 , а след цикличен

$$H = \sum_i \tilde{\mathbf{q}}_i \tilde{\mathbf{p}}_i^\top$$

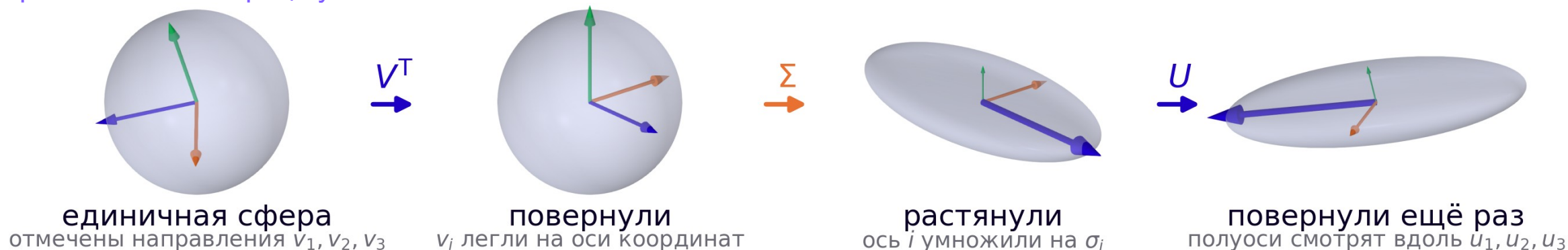
Н собрана из измерений: ни ортогональности, ни единичного определителя ей никто не обещал

Максимум линейной функции по кривому множеству поворотов



Что умеет произвольная матрица

Н уже собрана: девять чисел из измерений, и это не поворот. Чтобы вынуть из неё поворот, надо понять, что произвольная матрица умеет



$$A = U \Sigma V^T, \quad \Sigma = \text{diag}(\sigma_1 \geq \sigma_2 \geq \dots \geq 0)$$

Ортогональное преобразование, растяжение вдоль осей, ещё одно ортогональное

это и есть сингулярное разложение, SVD: существует для любой вещественной матрицы, в том числе прямоугольной

σ_i — сингулярные числа, столбцы U и V — левые и правые сингулярные векторы

U и V ортогональны, а не обязательно повороты: при $\det A < 0$ один из них — отражение



$$A v_i = \sigma_i u_i, \quad |\det A| = \sigma_1 \sigma_2 \sigma_3$$

v_i стоял на сфере — стал полуосью длины σ_i вдоль u_i

Ранг

число ненулевых σ_i
одно нулевое — блин, два — отрезок

Почти-ядро

вместо нуля на практике крошечное σ
 v_i при наименьшем σ_i почти
уничтожается

Обусловленность

σ_1 / σ_n — вытянутость эллипсоида
 10^k — потеря k значащих цифр из 16

Определитель вырожденность не измеряет

умножьте матрицу 3×3 на 100 — определитель вырастет в миллион раз, а σ_i — только в сто, и их отношение не изменится

$|\det A| = \sigma_1 \sigma_2 \sigma_3$, но σ_i неотрицательны: SVD не видит, переворачивает ли матрица ориентацию



$$H = \sum_i \tilde{\mathbf{q}}_i \tilde{\mathbf{p}}_i^\top = U \Sigma V^\top$$

$$\text{tr}(R^\top U \Sigma V^\top) = \text{tr}(\Sigma M) = \sum_i \sigma_i m_{ii} \leq \sigma_1 + \sigma_2 + \sigma_3, \quad M = V^\top R^\top U$$

M ортогональна, поэтому $|m_{ii}| \leq 1$, а σ_i неотрицательны

$$m_{ii} = 1 \Leftrightarrow M = I \Rightarrow R^\top = V U^\top \Rightarrow R = U V^\top$$

Весь алгоритм

вычесть центроиды из обоих облаков

$H = \sum \tilde{\mathbf{q}}_i \tilde{\mathbf{p}}_i^\top$, одно SVD матрицы 3×3

$R = U V^\top$, затем $\mathbf{t} = \bar{\mathbf{q}} - R \bar{\mathbf{p}}$

Чего в нём нет

итераций и начального приближения

шага обучения и критерия остановки

настроек, которые надо подбирать

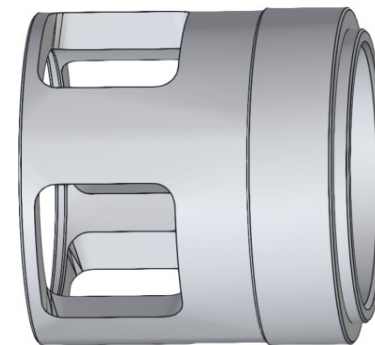
Ограничение оказалось не помехой, а тем, что задачу решило



$$R = U \operatorname{diag}(1, 1, \det(UV^T)) V^T$$

поправка стоит одну строку и берётся из той же выкладки

- $\det = +1$ — поворот, $\det = -1$ — отражение, а σ_i знака не помнят
- без поправки алгоритм вывернет деталь и не предупредит
- плоское расположение точек не вырождает задачу: поправка чинит знак



зеркало

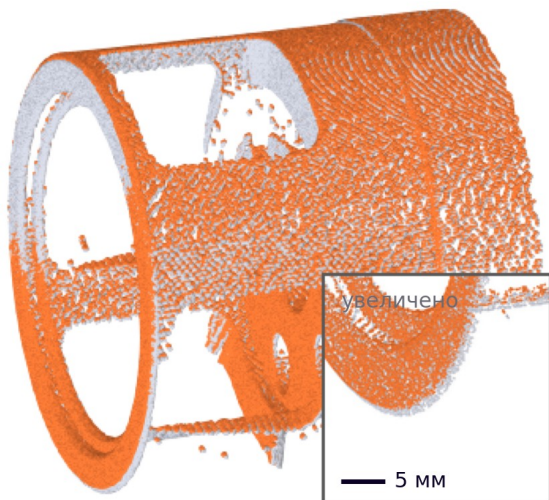
Вырождают коллинеарность и теснота, а не плоскость

пять точек на торцевом фланце — набор толщиной 0.2 мм против 30 мм у разнесённых: третье направление держится на шуме



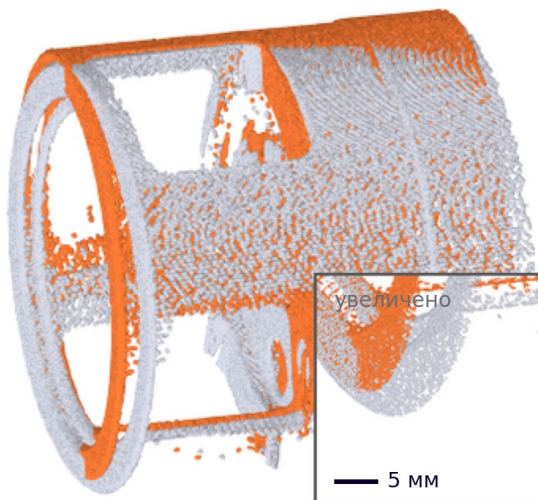
Для точных соответствий задача решена в замкнутой форме

серое — где деталь на самом деле, оранжевое — куда её поставило решение



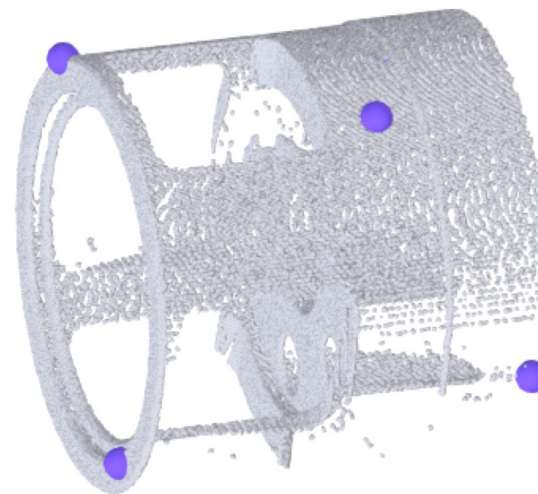
точки с шумом

шум 0.6 мм → мимо на 0.8 мм



одна пара неверна

мимо на 15 мм



и это всё, что участвовало

5 точек из 114 тысяч

Решение надо уточнять — малыми шагами по всему облаку сразу

а это уже другая задача, и решается она не той математикой



Точки с шумом

взять не пять точек, а все сто тысяч: разброс падает как $\sqrt{N}$
но для каждой нужна пара, а пар не размечали

Пар не размечали

назначить соседа ближайшим — и пересчитывать его после каждого шага
положение зависит от соседей, соседи от положения: круг рвётся итерацией

Часть соседей неверна

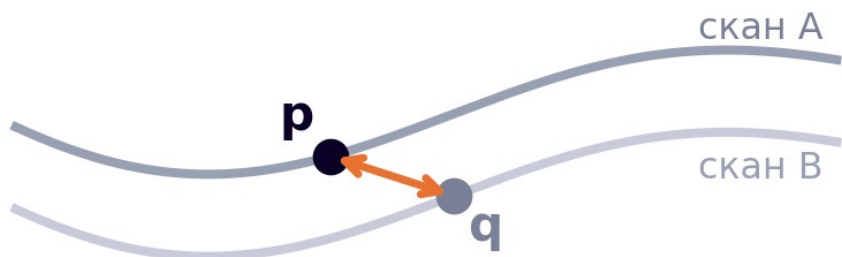
потеря не должна давать большим невязкам большой вес
и мерить надо до поверхности, а не до случайно легкой точки

Итерация здесь не способ решить уравнение, а способ разорвать круг

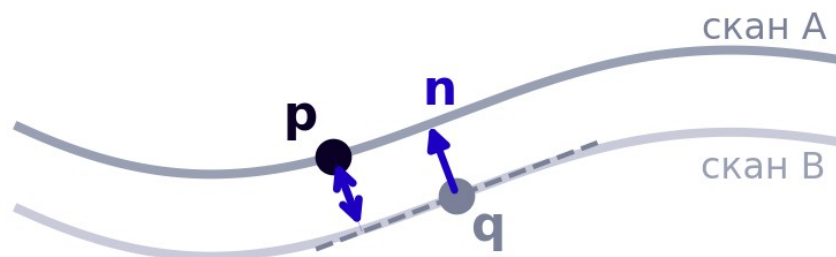


$\mathbf{n}$ — нормаль поверхности в точке соседа; вдоль стенки точка вольна скользить, это не ошибка

до самой точки соседа



до его касательной плоскости



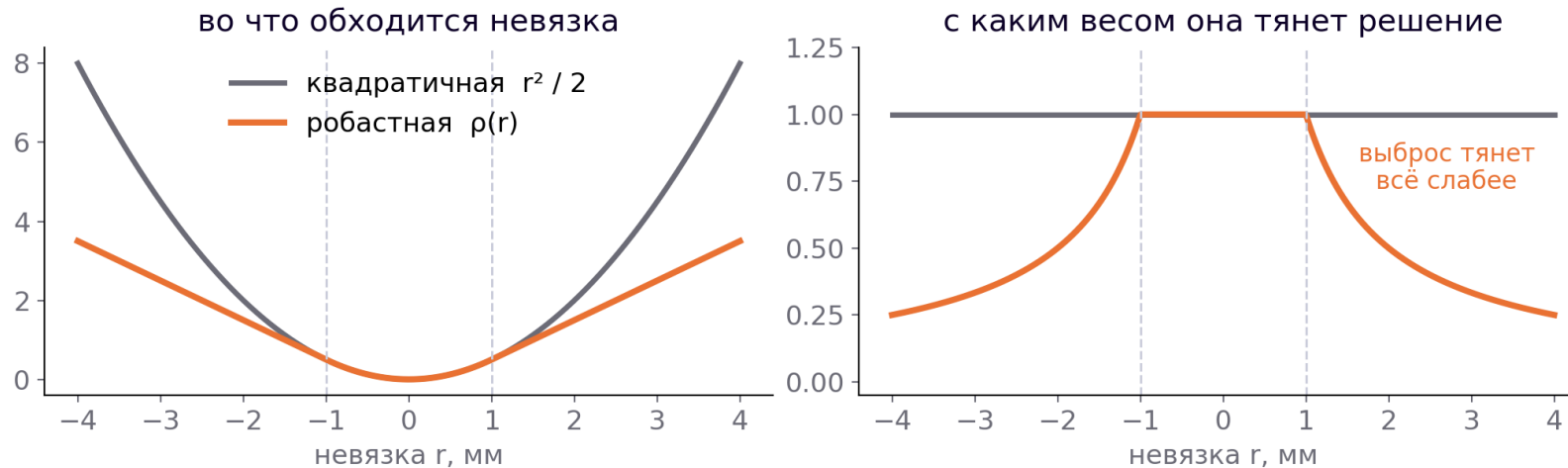
$r_i = \mathbf{n}_{j(i)}^\top (R\mathbf{p}_i + \mathbf{t} - \mathbf{q}_{j(i)})$ — расстояние до касательной плоскости

Скольжение вдоль стенки ошибкой не является

$\mathbf{n}$ — нормаль поверхности в точке соседа; считается по ковариации нескольких ближайших точек — то самое почти-ядро



невязка r — то самое расстояние до касательной плоскости соседа с прошлого слайда, в миллиметрах



Порог ставят по шуму сканера: что дальше — не измерение

неквадратичная потеря и убивает замкнутую форму: у Кабша весь вывод держался на раскрытии квадрата

на практике — итеративно перевзвешенные наименьшие квадраты



было

$$E(R, \mathbf{t}) = \sum_i \|R\mathbf{p}_i + \mathbf{t} - \mathbf{q}_i\|^2, \quad \text{пары } (\mathbf{p}_i, \mathbf{q}_i) \text{ заданы}$$

стало

$$E(R, \mathbf{t}) = \sum_i \rho(\mathbf{n}_{j(i)}^\top (R\mathbf{p}_i + \mathbf{t} - \mathbf{q}_{j(i)}))$$

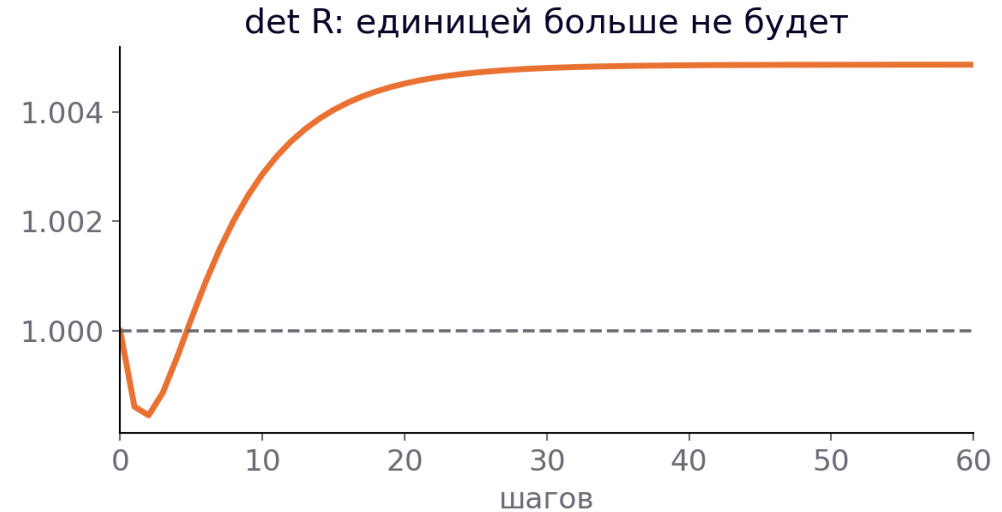
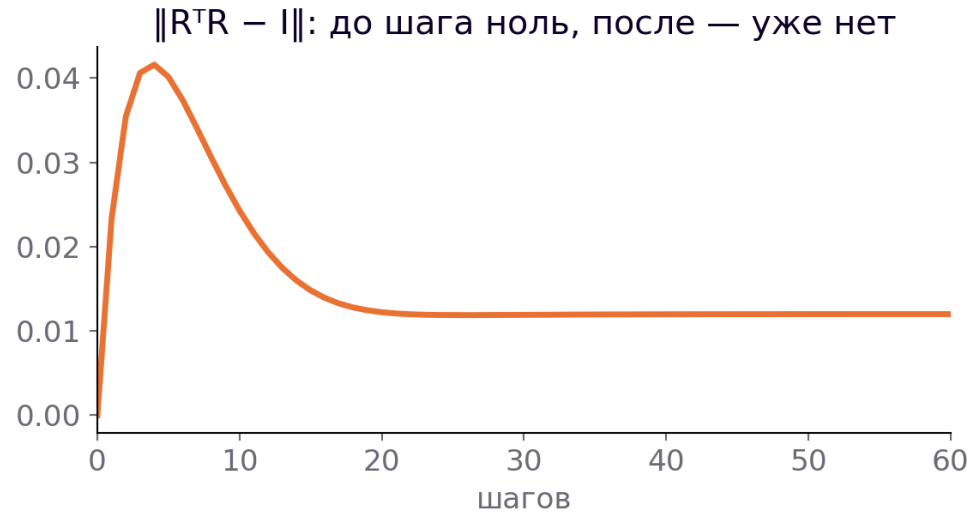
$$j(i) = \operatorname{argmin}_j \|R\mathbf{p}_i + \mathbf{t} - \mathbf{q}_j\| \leftarrow \text{зависит от } R, \mathbf{t}$$

ρ — робастная потеря, $\mathbf{n}$ — нормаль в точке соседа, $j(i)$ — сам сосед

Найти соседей при текущем положении, улучшить R и $\mathbf{t}$, повторить — это ICP

раскрыть скобки нельзя: при изменении R меняется и то, какие точки в них стоят





«шагнуть по R » на предыдущем слайде означает $R \leftarrow R - \eta \partial E / \partial R$. Функционал берём самый простой: поломка не в нём, а в способе шагать

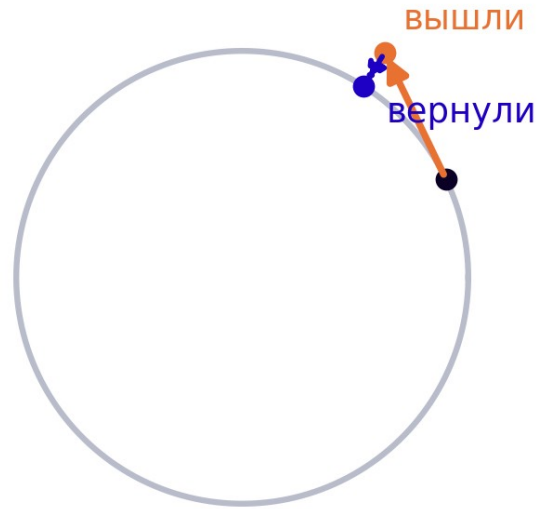
Вращения не образуют линейного пространства: складывать их нечем

и оптимизатор не жалуется: невязка честно падает, минимум по всем матрицам не больше минимума по поворотам

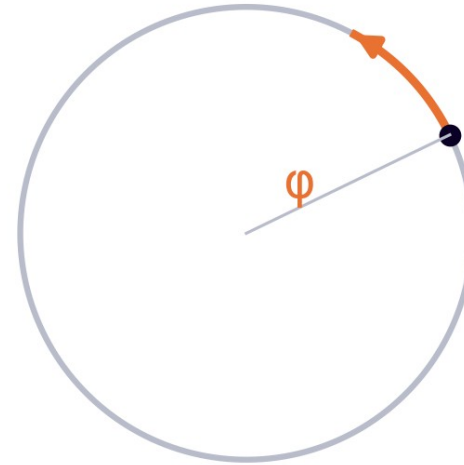


Чинить снаружи или считать внутри

шагнуть наружу и спроецировать



идти по самой окружности



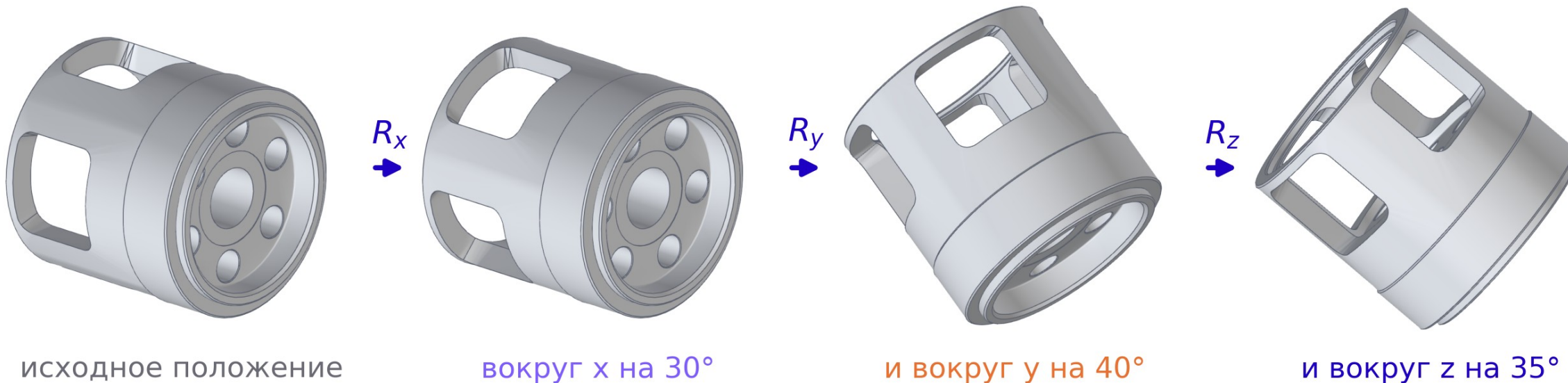
Ограничение надо унести внутрь параметризации

у окружности 2 числа – 1 уравнение = 1 своя координата; у вращений $9 - 6 = 3$, и любые три числа законны

проекция дифференцируема, так тоже можно. Но неизвестных остаётся девять при трёх степенях свободы, и шаг выходит не тем, что заказали



$$R(\alpha, \beta, \gamma) = R_z(\alpha) R_y(\beta) R_x(\gamma)$$

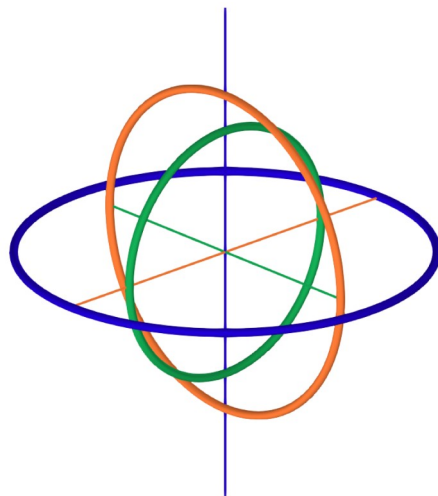


Ограничение исчезло: любая тройка углов — законный поворот

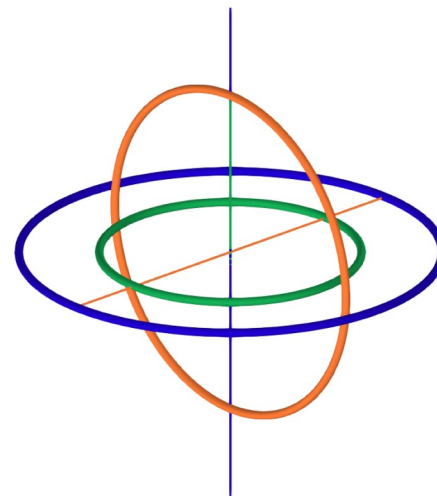
первый угол приводит ось в нужную плоскость, второй доводит до направления, третий доворачивает деталь вокруг неё

цена номер один: порядок и подвижность осей можно выбрать 24 способами, и все встречаются в коде





общее положение: три независимые оси



средний угол 90° : зелёная ось легла на синюю

$\beta = \pi/2$: $R(\alpha, \beta, \gamma)$ зависит только от $\alpha - \gamma$

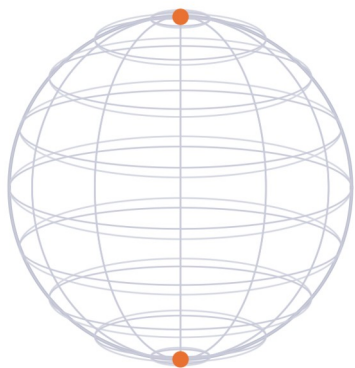
Ручек три, независимых движений два

само отображение углов в матрицу ведёт себя смирно; плохо обратной задаче: ради малого доворота углам приходится меняться на много



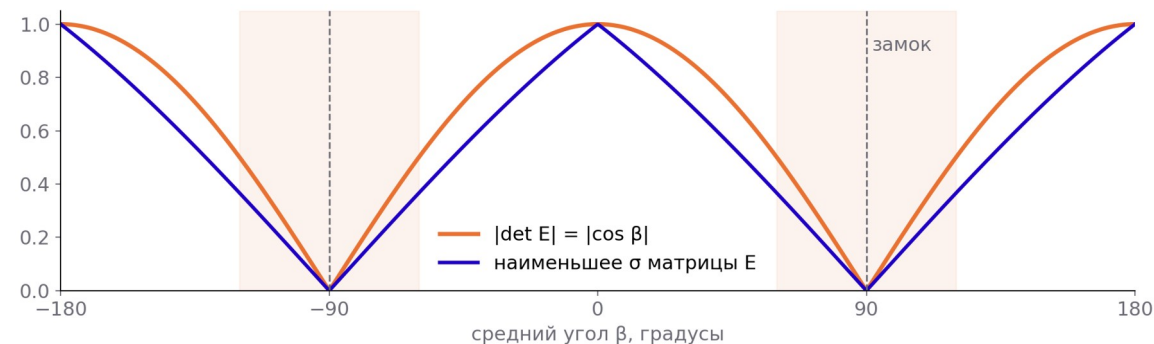
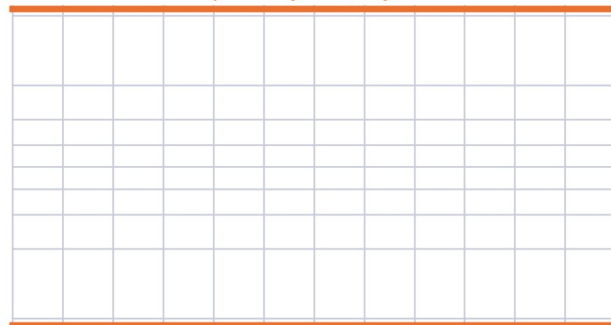
Дырку нельзя убрать, можно только переставить

глобус: полюса — обычные точки



плоская карта: точка стала линией

полюс растянут в целую линию



$$\omega = E(\beta, \gamma) (\dot{\alpha}, \dot{\beta}, \dot{\gamma})^T, \quad \det E = \cos \beta$$

Е переводит скорости трёх углов в угловую скорость детали; $\det E = 0$ значит, что часть направлений вращения углами недостижима

Замкнутую поверхность не положить на плоский лист без разрыва

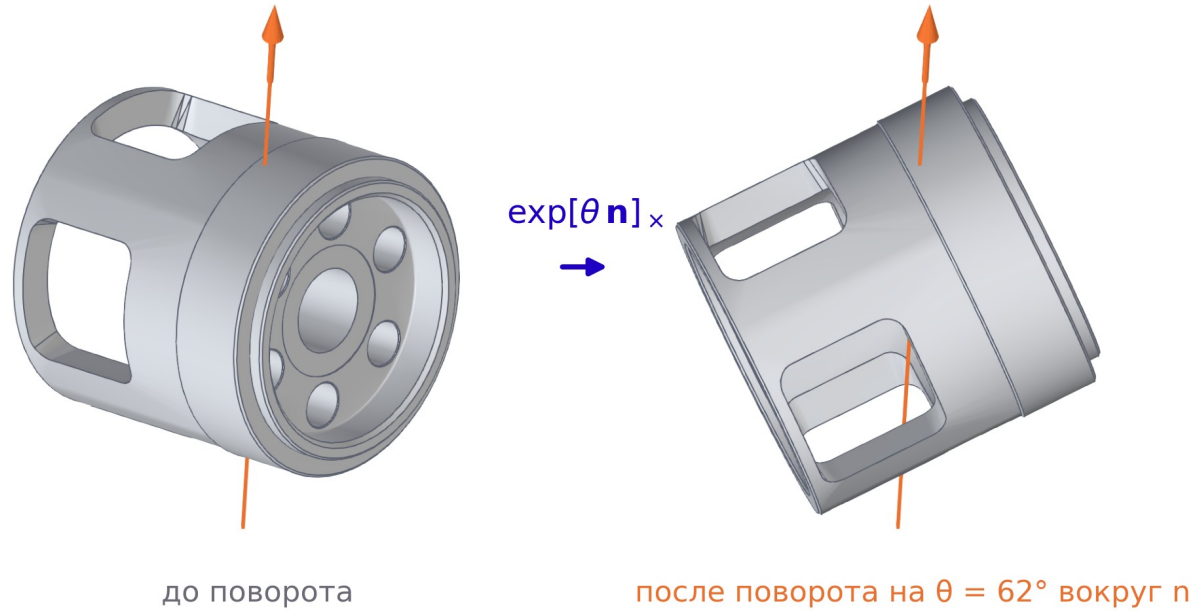
при $\beta = 60^\circ$ наименьшее σ матрицы Е уже 0.37: чтобы повернуть деталь на градус, углам надо измениться почти на три

дырку надо отправить на разворот — туда, где мы не работаем



Три числа вдоль самого поворота: ось и угол

$$\omega = \theta \mathbf{n}, \quad \|\mathbf{n}\| = 1, \quad \theta \in [0, \pi]$$



Теорема Эйлера: у любого поворота есть ось, которая остаётся на месте

осей комнаты в записи нет — значит, нет и конвенций; особенность уехала на разворот



$$\dot{\mathbf{x}} = \boldsymbol{\omega} \times \mathbf{x} = [\boldsymbol{\omega}]_{\times} \mathbf{x} \implies \mathbf{x}(t) = \exp(t [\boldsymbol{\omega}]_{\times}) \mathbf{x}(0)$$

скорость точки при вращении — векторное произведение $\boldsymbol{\omega}$ на неё саму, а его матрица кососимметрична

это линейное уравнение с постоянной матрицей, и его решение известно: экспонента

при $t = 1$ слева стоит поворот на угол $\|\boldsymbol{\omega}\|$ вокруг направления $\boldsymbol{\omega}$

Матрица поворота — экспонента кососимметричной матрицы

тот же факт, что и для скалярного $\dot{x} = ax$, только вместо числа матрица



$$\exp K = I + K + \frac{K^2}{2!} + \frac{K^3}{3!} + \dots, \quad [\mathbf{n}]_{\times}^3 = -[\mathbf{n}]_{\times}$$

куб кососимметричной матрицы единичной оси равен минус ей самой — значит, все степени сводятся к двум

$$R = I + \sin \theta [\mathbf{n}]_{\times} + (1 - \cos \theta) [\mathbf{n}]_{\times}^2$$

при матрице собирается ряд синуса, при её квадрате — ряд $1 - \cos$

$$R = \exp [\boldsymbol{\omega}]_{\times}, \quad \boldsymbol{\omega} = \log R, \quad [\boldsymbol{\omega}]_{\times}^{\top} = -[\boldsymbol{\omega}]_{\times}$$

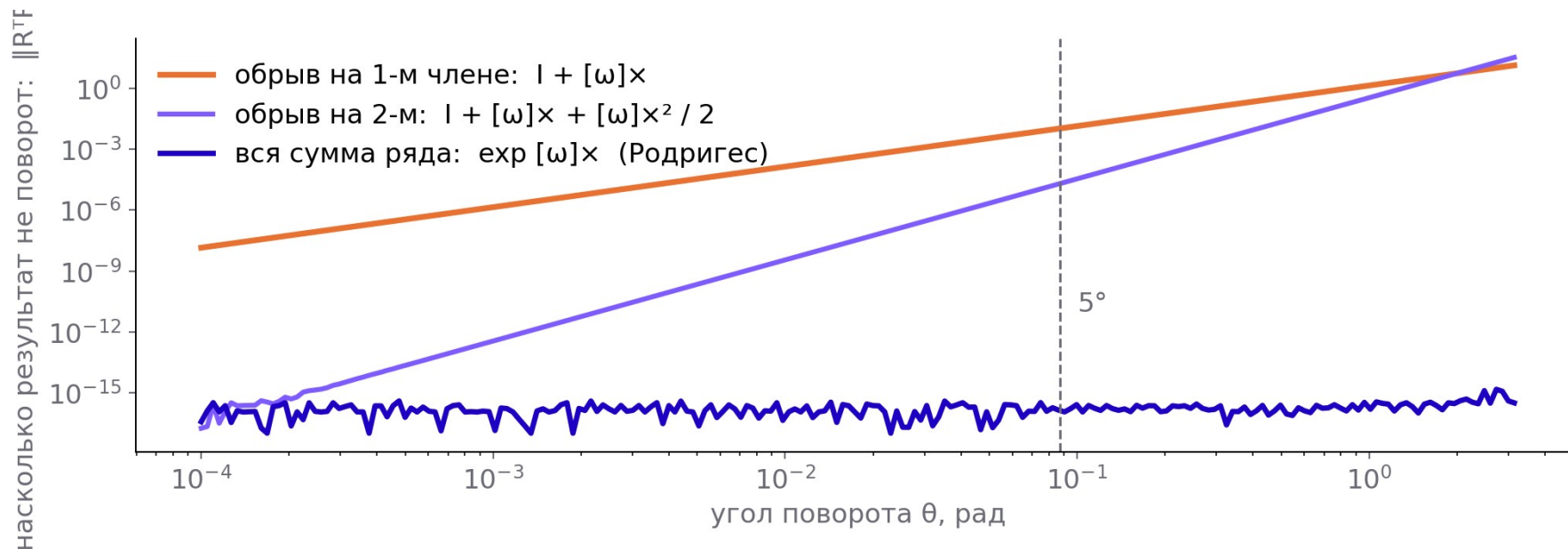
exp переводит три числа в поворот точно, log возвращает обратно

кососимметричные матрицы 3×3 — то самое касательное пространство: там складывать и умножать можно, и там мы будем считать поправки



Почему нельзя просто оборвать ряд

соблазн: при малом угле $\sin \theta \approx \theta$, а $1 - \cos \theta \approx 0$, значит, хватит первого члена



Обрыв даёт не приближённый поворот, а не-поворот

пять градусов — типичный шаг итерации; по шагам отклонение накапливается, и предупреждения не будет



$$q = (w, \mathbf{v}), \quad w \in \mathbb{R}, \quad \mathbf{v} \in \mathbb{R}^3 \text{ — четыре числа}$$

$$q = (\cos(\theta/2), \sin(\theta/2) \mathbf{n}) \longleftarrow \text{те же } \theta \text{ и } \mathbf{n}, \text{ что у оси-угла}$$

теорема Эйлера уже дала нам ось и угол — кватернион просто упаковывает их иначе

$$\mathbf{x}' = q(0, \mathbf{x})q^*, \quad q^* = (w, -\mathbf{v})$$

применяется дважды, поэтому в четвёрке стоит половина угла: каждое умножение поворачивает на половину

Четыре числа, одно уравнение $\|q\| = 1$ — те же три степени свободы

q и $-q$ — один и тот же поворот: прибавьте к θ полный оборот, и вся четвёрка сменит знак. Это двойное покрытие



$$q_1 q_2 = (w_1 w_2 - \mathbf{v}_1 \cdot \mathbf{v}_2, \quad w_1 \mathbf{v}_2 + w_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2)$$

из векторного произведения умножение некоммутативно — как и повороты

Чинить ограничение

за 200 000 композиций матрица уходит на $9e-03$,
кватернион на $9e-04$
матрице нужно разложение, кватерниону — деление на
длину

Сцеплять и хранить

16 умножений против 27, четыре числа против девяти
для одной операции неважно, для графа из тысяч поз —
важно

Ограничение одно и скалярное — поэтому чинится делением

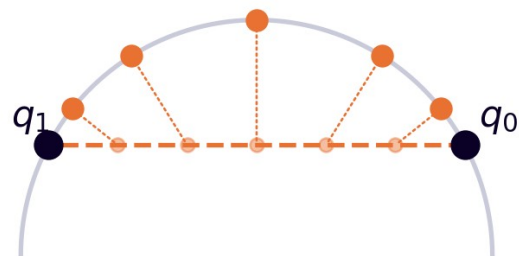
и третья причина, которой хватит на два слайда: интерполяция



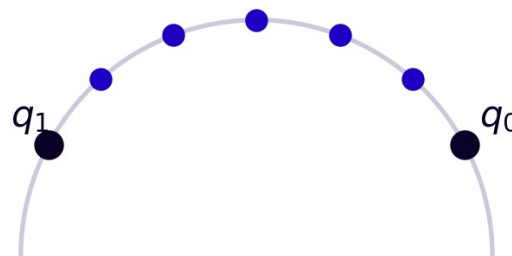
Что такое slerp

единичные кватернионы лежат на сфере — интерполируем точки на ней

линейно: по хорде, потом нормировать



slerp: по дуге между ними



хорда короче радиуса, поэтому нормировка
сдвигает точки неравномерно

$$\text{slerp}(q_0, q_1; t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} q_0 + \frac{\sin(t\Omega)}{\sin \Omega} q_1, \quad \cos \Omega = q_0 \cdot q_1$$

Идти по дуге между двумя точками сферы, а не по хорде внутри неё

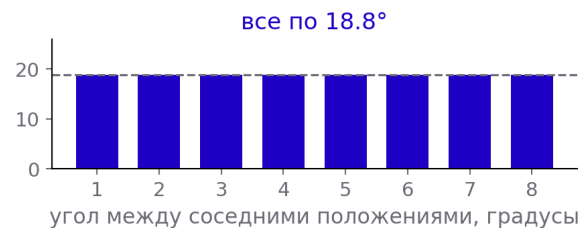
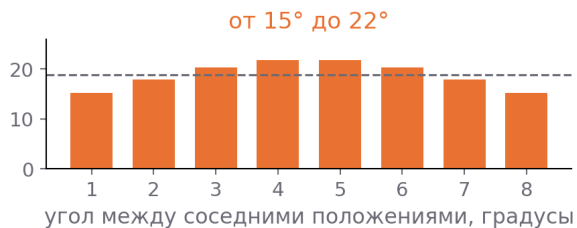
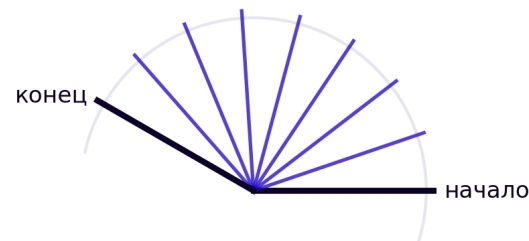
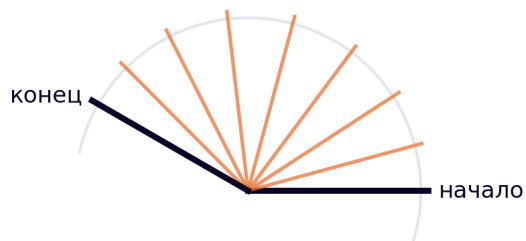
та же развилка, что и на слайде про проекцию: шагнуть наружу и вернуть или идти по самой поверхности



9 промежуточных положений между двумя ориентациями, развёрнутыми на 150°

усреднить покомпонентно и нормировать

идти по дуге: slerp



Разброс углов — это неравномерная угловая скорость

при больших углах разница видна глазом, при малых её нет

ловушка: при несогласованных знаках дуга получится длинной — лечится сменой знака одной четвёрки



применить к точкам

матрица 3×3

хранить и интерполировать

кватернион

оптимизировать

приращение ω , применяется через `exp`

показать человеку

углы Эйлера

Ответ диктует глагол, а не объект

и направление держите в имени переменной — для всех четырёх записей



текущие R и t уже примерно верны — их дал Кабш. Ищем не их заново, а маленькую поправку

$$R \leftarrow \exp[\delta\varphi] \times R, \quad t \leftarrow \exp[\delta\varphi] \times t + \delta\rho$$

поворот умножаем на $\exp$, сдвиг прибавляем — из блока D

$\delta = (\delta\varphi, \delta\rho) \in \mathbb{R}^6$ три числа на поворот, три на сдвиг

Шесть свободных чисел: ограничение спрятано в экспоненте

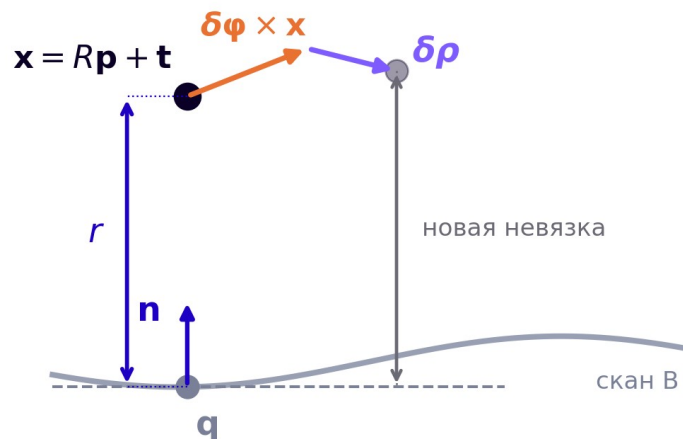
после применения поправка вкатывается в R и t , а δ обнуляется — на следующем шаге мы снова стоим в нуле

поэтому особенность при развороте до нас никогда не дотягивается



$\mathbf{x}_i = R\mathbf{p}_i + \mathbf{t}$, $r_i = \mathbf{n}_i^\top (\mathbf{x}_i - \mathbf{q}_i)$ текущее положение и текущая невязка

поправка сдвигает точку на $\delta\varphi \times \mathbf{x}$ от поворота и на $\delta\rho$ от переноса; невязка — проекция на $\mathbf{n}$



$$r_i(\boldsymbol{\delta}) = \mathbf{n}_i^\top (\mathbf{x}_i + \delta\varphi \times \mathbf{x}_i + \delta\rho - \mathbf{q}_i)$$

Меняется не всё смещение, а только его проекция на нормаль

составляющая вдоль поверхности невязку не меняет — ровно ради этого мы и взяли расстояние до плоскости



$$r_i(\boldsymbol{\delta}) = r_i + (\mathbf{x}_i \times \mathbf{n}_i)^\top \boldsymbol{\delta}\varphi + \mathbf{n}_i^\top \boldsymbol{\delta}\rho$$

смешанное произведение $\mathbf{n} \cdot (\delta\varphi \times \mathbf{x})$ переставили по кругу так, чтобы $\delta\varphi$ оказалась справа

$$\mathbf{J}_i = \left((\mathbf{x}_i \times \mathbf{n}_i)^\top, \mathbf{n}_i^\top \right) \mathbf{x}_i = R\mathbf{p}_i + \mathbf{t}$$

шесть чисел на одну точку: три от поворота, три от сдвига. Собираем такие строки в матрицу $\mathbf{J}$

$$r_i(\boldsymbol{\delta}) \approx r_i + \mathbf{J}_i \boldsymbol{\delta}, \quad \boldsymbol{\delta} = (\boldsymbol{\delta}\varphi, \boldsymbol{\delta}\rho) \in \mathbb{R}^6$$

Обрывать ряд в модели можно, в самом шаге — нельзя

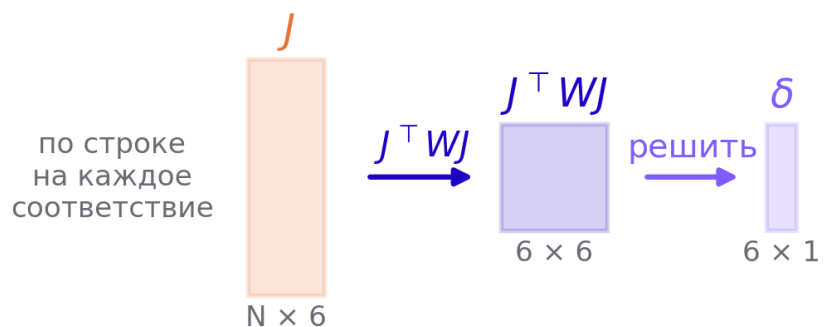
при достаточно малом шаге ошибку модели исправит следующая итерация; сам шаг применяется точно, поэтому не накапливается

в $\mathbf{J}$ входит текущее $\mathbf{x}$, поэтому $\mathbf{J}$ пересчитывают на каждой итерации



$$(J^{\top} W J + \lambda I) \delta = -J^{\top} W \mathbf{r}, \quad w_i = \rho'(r_i) / r_i$$

W — диагональ из весов прошлого слайда: без неё они ни на что не влияли бы. λ гасит слабо определённые направления



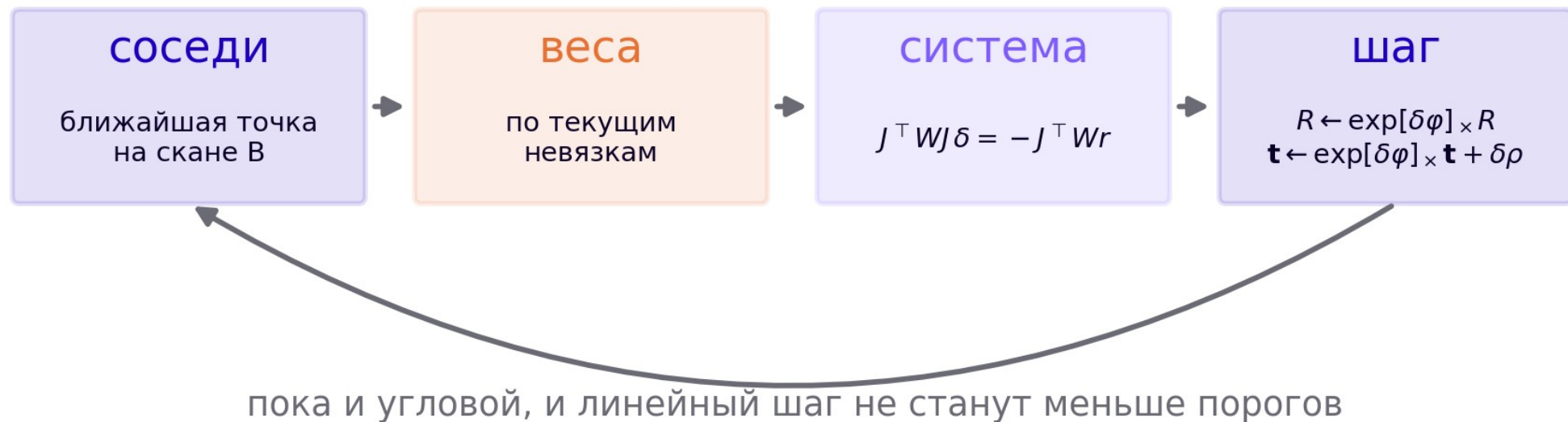
сколько бы точек ни было, решается система из шести

Сто тысяч уравнений сворачиваются в систему из шести

тот же фокус, что у Кабша: данные входят только через сумму, поэтому после сборки системы точки больше не нужны

сингулярные числа этой матрицы 6×6 скажут, определена ли задача: на плоском облаке три из них будут около нуля





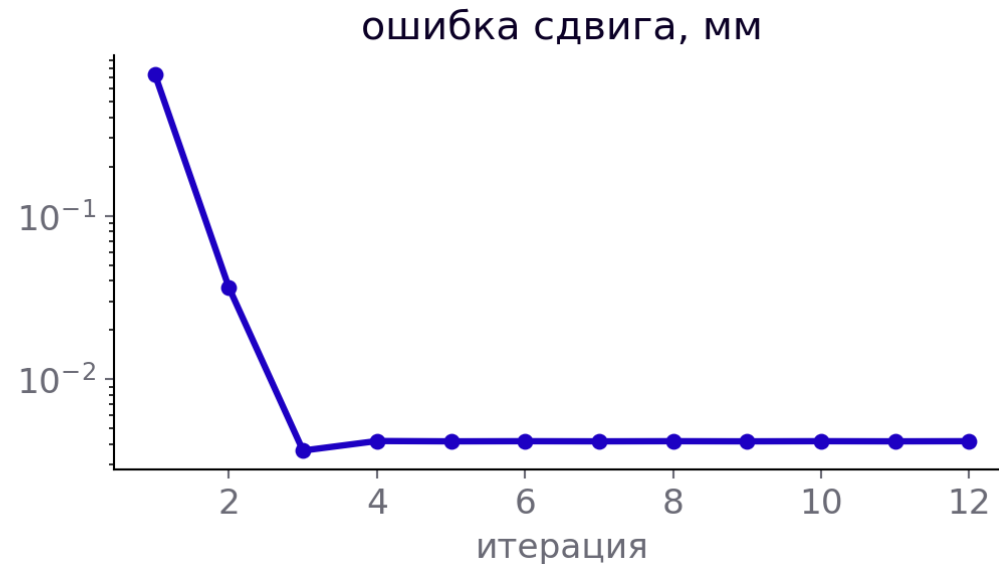
Цикл соседи → веса → система → шаг: каждый блок выведен по нужде

соседи — из того, что пар не размечали; веса — из того, что часть соседей назначена неверно

менять можно любой блок: замените невязку на разность пикселей — получите задачу лекции 3



Что получается на нашей детали



старт 6° и 4 мм, 40 тысяч точек, шум сканера — за 12 итераций до 0.014° и 0.004 мм

Полка на графике — это уровень шума, а не недоработка

и всё это работает только потому, что стартовали близко: при большом развороте соседи назначаются неверно, и итерация сходится не туда



Сканов десяток

тринадцать попарных совмещений
накопят ошибку

искать все позы сразу: 6 чисел на
скан

Другая невязка

разность пикселей вместо расстояния
до плоскости

цикл соседи → веса → система →
шаг остаётся тем же

Разреженность

каждая точка видна с нескольких
камер

на ней стоит bundle adjustment

Всё это в библиотеках называется работой с группами Ли

приращение в касательном пространстве, $\exp$ и $\log$ — стандартный интерфейс в *ceres*, *g2o*, *GTSAM*, *Sophus*

теперь, открывая их документацию, вы знаете, о чём там речь и почему оно устроено именно так



Что мы держали запертым

сегодня

что добавляет



(0 0 0 1)

жёсткое движение

6 чисел

поворот и сдвиг

сохраняет длины и углы



(0 0 0 1)

подобие

7 чисел

+ общий масштаб

сохраняет углы и отношения длин



(0 0 0 1)

аффинное

12 чисел

+ сжатие по осям, скос, отражение

сохраняет параллельность



проективное

15 чисел

+ перспективное искажение

сохраняет прямизну: прямые остаются прямыми

Отпертая степень свободы — это то, чего сенсор не различает

масштаб нам дал прибор, а не математика: вдвое большая сцена, снятая вдвое дальше, даёт тот же снимок

запирание клетки — всегда дополнительная информация о мире, которую надо откуда-то взять



сегодня невязка — расстояние между точкой и поверхностью, в миллиметрах

$$\mathbf{r} = \pi(K [R | \mathbf{t}] \mathbf{X}) - \mathbf{u} \text{ — та же невязка, но в пикселях}$$

Что здесь что

$\mathbf{X}$ — точка детали в пространстве, в однородных координатах

$[R | \mathbf{t}]$ — наше сегодняшнее преобразование: переводит точку в систему камеры

K — внутренние параметры камеры: фокусное расстояние и центр снимка, в пикселях

π — деление на третью координату, то самое деление из однородных координат

$\mathbf{u}$ — пиксель, в котором точку реально видно на снимке

Меняется одна строка: чем считается невязка

$[R | \mathbf{t}]$ внутри остаётся жёстким движением: деление на глубину в нём ничего не отпирает

вниз по таблице сцену уводит неизвестная глубина: по снимкам одной камеры она определена лишь до подобия



чем записать

матрица 4×4 , композиция — умножение

как найти

Кабш: центрировать, H, SVD, знак определителя

чем записывать вращение

приращение ω , применяется через $\exp$

как уточнить

кольцо: соседи, веса, система 6×6 , точный шаг

Единственная лекция курса, где ничего не реконструируется

зато всё, что здесь появилось, работает дальше каждую неделю: внутри любого метода восстановления стоит та же матрица и то же кольцо

